Chapitre : Le langage Dart

Types, fonctions, boucles et classes

Pr. Abdelfattah Abassi

Département Physique Appliquée – Laboratoire LAVETE
Faculté des Sciences et Techniques de Settat
Université Hassan I

Année universitaire 2026–2027

Objectifs et organisation du chapitre

À la fin de ce chapitre, vous saurez

- écrire et exécuter un programme Dart ;
- utiliser les **types** de données et les variables ;
- prendre des décisions (`if`, `switch`) ;
- répéter des actions avec des **boucles** ;
- écrire des **fonctions** réutilisables ;
- modéliser avec des **classes** et des objets.

Prérequis

Aucun. Nous partons de zéro : pas besoin d'avoir déjà programmé.

Méthode

Pour chaque notion : *idée* → *exemple* → *exercice*.

Tapez et testez chaque exemple vous-même !

Plan du chapitre

1. Les fonctions

2. Classes et objets

3. Synthèse et mini-projet

Plan du chapitre

1. Les fonctions

2. Classes et objets

3. Synthèse et mini-projet

Pourquoi écrire des fonctions ?

Une **fonction** est un bloc de code **nommé** qu'on peut appeler autant de fois que l'on veut.

Analogie : la machine à café

On donne des ingrédients (**paramètres**), on appuie sur un bouton (**appel**), on récupère un café (**valeur retournée**).

Avantages

- **Réutilisation** : écrire une fois, appeler partout
- **Lisibilité** : un nom clair décrit l'action
- **Maintenance** : une seule correction à faire
- **Tests** plus faciles

Anatomie d'une fonction

```dart
int addition(int a, int b) {
  int resultat = a + b;
  return resultat;
}

void main() {
  int s = addition(3, 4);
  print(s);           // 7
  print(addition(10, 5)); // 15
}
```

`int` (devant)	type de **retour**
`addition`	**nom** de la fonction
`(int a, int b)`	**paramètres**
`{ ... }`	**corps**
`return`	renvoie la valeur

Paramètre ou argument ?

Paramètre : variable dans la définition (`a`, `b`).
Argument : valeur passée à l'appel (`3`, `4`).

Fonction qui ne renvoie rien : `void`

```dart
void saluer(String nom) {
  print('Bonjour $nom !');
}

void main() {
  saluer('Sara');
  saluer('Youssef');
}
```

Sortie :

```
Bonjour Sara !
Bonjour Youssef !
```

`void` ou valeur ?

`void` : la fonction **fait** quelque chose (affiche, modifie...).

Type de retour (`int`, `String`...) : la fonction **calcule** et renvoie un résultat.

Bonne pratique

Une fonction = **une seule tâche**, avec un nom **verbe** : `calculerMoyenne`, `estPair`...

Syntaxe fléchée =>

Quand le corps est **une seule expression**, on peut l'écrire plus court :

```
int carre(int x) {              // forme longue
  return x * x;
}

int carre2(int x) => x * x;     // forme fléchée (équivalente)

bool estPair(int n) => n % 2 == 0;
double moyenne(double a, double b) => (a + b) / 2;

void main() {
  print(carre(5));              // 25
  print(estPair(7));            // false
  print(moyenne(12, 15));       // 13.5
}
```

Paramètres positionnels optionnels

```dart
String saluer(String nom, [String titre = 'M.']) {
  return '$titre $nom';
}

void main() {
  print(saluer('Alami'));          // M. Alami
  print(saluer('Alami', 'Pr.'));   // Pr. Alami
}
```

- Entre `[ ]` : paramètres **facultatifs**.
- On leur donne une **valeur par défaut** (`= 'M.'`).
- L'ordre des arguments compte.

Paramètres nommés

```dart
double prixTTC({required double prixHT, double tva = 0.25}) {
  return prixHT * (1 + tva);
}

void main() {
  print(prixTTC(prixHT: 100));              // 125.0
  print(prixTTC(tva: 0.5, prixHT: 100));    // 150.0
}
```

- Entre `{ }` : paramètres **nommés**.
- À l'appel : `nom: valeur`, dans **n'importe quel ordre**.
- `required` : obligatoire.
- Sinon : valeur par défaut ou type nullable (`int?`).

Flutter

Les widgets Flutter utilisent massivement les paramètres nommés.

Portée des variables

```dart
int global = 10;              // visible partout

void test() {
  int local = 5;              // visible ici seulement
  print(global + local);      // 15
}

void main() {
  test();
  print(global);              // 10
  // print(local);            // ERREUR : inconnue ici
  for (int i = 0; i < 2; i++) { }
  // print(i);                // ERREUR : i n'existe plus
}
```

Règle

Une variable n'existe que dans le **bloc** `{ ... }` où elle est déclarée.

Conseil

Limitez les variables globales : préférez passer des **paramètres**.

Les fonctions sont des valeurs

```dart
int carre(int x) => x * x;

// une fonction reçoit une autre fonction
int appliquer(int x, int Function(int) f) => f(x);

void main() {
  print(appliquer(5, carre));          // 25
  print(appliquer(5, (n) => n * 2));   // 10  (anonyme)

  var triple = (int n) => n * 3;       // stockée
  print(triple(4));                    // 12
}
```

Fonction anonyme

Fonction **sans nom**, écrite sur place :
`(paramètres) => expression`

Pourquoi ?

Indispensable pour les **callbacks** Flutter : `onPressed: () { ... }`

Traiter une liste : map, where, reduce

```dart
var notes = [8, 12, 15, 6, 17];

var admis = notes.where((n) => n >= 10).toList();
print(admis);                  // [12, 15, 17]

var bonus = notes.map((n) => n + 1).toList();
print(bonus);                  // [9, 13, 16, 7, 18]

int total = notes.reduce((a, b) => a + b);
print(total);                  // 58

print(notes.every((n) => n >= 10));  // false
print(notes.any((n) => n >= 16));    // true
```

`where`	**filtre**
`map`	**transforme**
`reduce`	**combine**
`every`	tous vrais ?
`any`	au moins un ?

`.toList()` convertit le résultat en liste.

La récursivité

Une fonction **récursive** s'appelle elle-même.

```
int factorielle(int n) {
  if (n <= 1) {
    return 1;                    // cas de base
  }
  return n * factorielle(n - 1);
}

void main() {
  print(factorielle(5));         // 120
}
```

Déroulement :

```
fact(5) = 5 * fact(4)
fact(4) = 4 * fact(3)
fact(3) = 3 * fact(2)
fact(2) = 2 * fact(1)
fact(1) = 1    → 120
```

Obligatoire

Un **cas de base** qui arrête la récursion, sinon : dépassement de pile.

Exercices : fonctions

1. Écrivez `double celsiusVersFahrenheit(double c)` ($F = C \times 1{,}8 + 32$).
2. Écrivez `bool estPremier(int n)` (utilisez une boucle).
3. Écrivez `String mention(double moyenne)` (réutilisez le `if / else if`).
4. Écrivez une fonction avec paramètres nommés `creerMessage({required String nom, int age = 18})`.
5. Avec `where` et `map`, à partir de `[3, 8, 12, 5, 20]`, obtenez les carrés des nombres supérieurs à 5.
6. Écrivez `int fibonacci(int n)` de façon récursive.

Plan du chapitre

1. Les fonctions

2. Classes et objets

3. Synthèse et mini-projet

Pourquoi des classes ?

Sans classe

```
String nom1 = 'Sara';
int age1 = 20;
String nom2 = 'Youssef';
int age2 = 21;
// et pour 100 étudiants ?
```

Avec une classe

On regroupe les **données** (nom, âge) et les **actions** (se présenter, vieillir) dans un seul modèle : `Etudiant`.

C'est la **programmation orientée objet** (POO).

Classe et objet

Classe

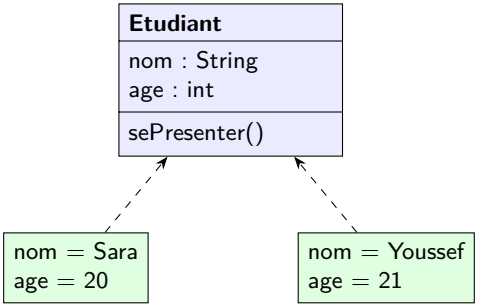
Un **plan** (un moule) : décrit les **attributs** et les **méthodes**.

Objet (instance)

Un exemplaire **concret** créé à partir de la classe, avec ses propres valeurs.

Analogie

Plan d'une maison = classe
Maisons construites = objets



Ma première classe

```dart
class Etudiant {
  String nom = '';
  int age = 0;

  void sePresenter() {
    print('Je suis $nom, $age ans.');
  }
}

void main() {
  Etudiant e1 = Etudiant();   // création d'un objet
  e1.nom = 'Sara';
  e1.age = 20;
  e1.sePresenter();
}
```

Sortie :

```
Je suis Sara, 20 ans.
```

- `class` : définit la classe (nom en **UpperCamelCase**).
- Les **attributs** sont des variables de la classe.
- Les **méthodes** sont des fonctions de la classe.
- `objet.attribut` : accès avec le point.

Le constructeur

Un **constructeur** initialise l'objet dès sa création.

```dart
class Etudiant {
  String nom;
  int age;

  Etudiant(this.nom, this.age);   // raccourci

  void sePresenter() =>
      print('Je suis $nom, $age ans.');
}

void main() {
  var e = Etudiant('Youssef', 21);
  e.sePresenter();
}
```

Forme équivalente (longue)

```dart
Etudiant(String nom, int age) {
  this.nom = nom;
  this.age = age;
}
```

`this` désigne **l'objet courant**. `this.nom` est l'attribut, `nom` seul est le paramètre.

Constructeurs : paramètres nommés et constructeurs nommés

```dart
class Etudiant {
  final String nom;
  final int age;
  String filiere;

  // paramètres nommés avec valeur par défaut
  Etudiant({required this.nom, required this.age, this.filiere = 'MST'});

  // constructeur nommé
  Etudiant.invite() : nom = 'Invité', age = 0, filiere = 'Aucune';
}

void main() {
  var e1 = Etudiant(nom: 'Imane', age: 22);
  var e2 = Etudiant.invite();
  print('${e1.nom} - ${e1.filiere}');   // Imane - MST
  print('${e2.nom} - ${e2.filiere}');   // Invité - Aucune
}
```

`final` : l'attribut ne peut plus être modifié après la création.

Encapsulation : attributs privés

```dart
class CompteBancaire {
  final String titulaire;
  double _solde = 0;              // privé (préfixe _)

  CompteBancaire(this.titulaire);

  double get solde => _solde;    // lecture seule
  void deposer(double montant) {
    if (montant > 0) _solde += montant;
  }

  bool retirer(double montant) {
    if (montant > 0 && montant <= _solde) {
      _solde -= montant;
      return true;
    }
    return false;
  }
}
```

```dart
var c = CompteBancaire('Sara');
c.deposer(500);
print(c.retirer(200)); // true
print(c.solde);        // 300.0
print(c.retirer(1000));// false
```

Idée

On **protège** les données : on ne les modifie que par des méthodes qui vérifient les règles.

En Dart, _ rend le membre privé à la **bibliothèque** (le fichier), pas seulement à la classe.

Getters et setters

```dart
class Etudiant {
  String nom;
  int _age;

  Etudiant(this.nom, this._age);

  int get age => _age;

  set age(int valeur) {
    if (valeur >= 0 && valeur < 120) {
      _age = valeur;
    }
  }

  // getter calculé
  bool get estMajeur => _age >= 18;
}
```

```dart
var e = Etudiant('Sara', 17);
print(e.estMajeur);  // false
e.age = 18;          // setter
print(e.estMajeur);  // true
e.age = -5;          // refusé
print(e.age);        // 18
```

Remarque

On utilise un getter/setter **comme un attribut** (sans parenthèses).

Membres statiques et toString

```dart
class Etudiant {
  static int compteur = 0;        // partagé par tous
  final String nom;
  final int age;

  Etudiant(this.nom, this.age) {
    compteur++;
  }

  @override
  String toString() => 'Etudiant($nom, $age ans)';
}

void main() {
  var a = Etudiant('Sara', 20);
  var b = Etudiant('Youssef', 21);
  print(a);                       // Etudiant(Sara, 20 ans)
  print(Etudiant.compteur);       // 2
}
```

static

Appartient à **la classe**, pas à un objet : on l'appelle avec le **nom de la classe**.

toString()

Définit comment l'objet s'affiche.
Sans elle : `Instance of 'Etudiant'`.

L'héritage

```dart
class Personne {
  String nom;
  Personne(this.nom);
  void sePresenter() {
    print("Je m'appelle $nom.");
  }
}

class Etudiant extends Personne {
  String filiere;
  Etudiant(String nom, this.filiere) : super(nom);

  @override
  void sePresenter() {
    super.sePresenter();           // appel au parent
    print('Je suis en $filiere.');
  }
}
```

```dart
var e = Etudiant('Sara', 'MST');
e.sePresenter();
```

Sortie :

```
Je m'appelle Sara.
Je suis en MST.
```

- `extends` : **hérite** des attributs et méthodes.
- `super` : accès au parent.
- `@override` : redéfinit une méthode.

Classes abstraites et polymorphisme

```dart
abstract class Forme {
  double aire();                  // sans corps
}

class Cercle extends Forme {
  final double r;
  Cercle(this.r);
  @override
  double aire() => 3.14159 * r * r;
}

class Rectangle extends Forme {
  final double l, h;
  Rectangle(this.l, this.h);
  @override
  double aire() => l * h;
}
```

```dart
List<Forme> formes = [
  Cercle(2),
  Rectangle(3, 4),
];
for (Forme f in formes) {
  print('Aire = ${f.aire()}');
}
```

Sortie :

```
Aire = 12.56636
Aire = 12.0
```

Polymorphisme : un même appel (`aire()`) donne un comportement différent selon l'objet.

Les énumérations (enum)

```dart
enum Niveau { licence, master, doctorat }

void main() {
  Niveau n = Niveau.master;
  print(n);           // Niveau.master
  print(n.name);      // master

  String texte = switch (n) {
    Niveau.licence  => 'Bac+3',
    Niveau.master   => 'Bac+5',
    Niveau.doctorat => 'Bac+8',
  };
  print(texte);       // Bac+5
}
```

Intérêt

Un **ensemble fini** de valeurs nommées : moins d'erreurs qu'avec des textes libres comme `'mster'`.

Exemples

Statut d'une commande, jours de la semaine, type de capteur IoT...

Exercices : classes

1. Créez une classe `Livre` (titre, auteur, nbPages) avec constructeur et méthode `afficher()`.
2. Ajoutez un getter `estLong` qui vaut `true` si `nbPages > 300`.
3. Créez `Rectangle` avec `aire()` et `perimetre()` ; testez avec deux objets.
4. Créez `Animal` (nom, méthode `crier()`) puis `Chien` et `Chat` qui redéfinissent `crier()`. Stockez-les dans une `List<Animal>`.
5. Ajoutez à `CompteBancaire` une méthode `virer(CompteBancaire autre, double m)`.

Plan du chapitre

1. Les fonctions

2. Classes et objets

3. Synthèse et mini-projet

Mini-projet : gestion des notes d'une classe

Cahier des charges

- Classe `Etudiant` : `nom` et liste de `notes`.
- `ajouterNote(double)` : refuse les notes hors de $[0\,;\,20]$.
- Getter `moyenne` et getter `mention`.
- `toString()` : « Sara : 16.25 (Très bien) ».
- Dans `main` : créer 3 étudiants, leur ajouter des notes, **trier** par moyenne décroissante, afficher le classement, la moyenne de la classe et le meilleur étudiant.

Notions utilisées

Types, conditions, boucles, collections, fonctions anonymes, classe, getters, `toString()`.

Mini-projet : la classe `Etudiant` (1/2)

```dart
class Etudiant {
  final String nom;
  final List<double> notes = [];
  Etudiant(this.nom);
  void ajouterNote(double note) {
    if (note >= 0 && note <= 20) {
      notes.add(note);
    } else {
      print('Note invalide : $note');
    }
  }
  double get moyenne {
    if (notes.isEmpty) return 0;
    double somme = 0;
    for (final n in notes) {
      somme += n;
    }
    return somme / notes.length;
  }
```

- `notes` : liste de `double` initialisée vide.
- `ajouterNote` : **condition** `&&` pour valider.
- `moyenne` : **boucle** avec accumulateur, cas de la liste vide traité.

La classe se poursuit sur la diapositive suivante.

Mini-projet : la classe `Etudiant` (2/2)

```dart
class Etudiant {
  // ... attributs, ajouterNote, moyenne

  String get mention {
    final m = moyenne;
    if (m >= 16) return 'Très bien';
    if (m >= 14) return 'Bien';
    if (m >= 12) return 'Assez bien';
    if (m >= 10) return 'Passable';
    return 'Insuffisant';
  }

  @override
  String toString() =>
      '$nom : ${moyenne.toStringAsFixed(2)} ($mention)';
}
```

- `mention` : getter qui réutilise `moyenne`.
- Les `return` successifs remplacent le `else if`.
- `toString()` : **interpolation** et arrondi à 2 décimales.

Mini-projet : le programme principal (1/2)

```dart
void main() {
  final classe = [
    Etudiant('Sara'),
    Etudiant('Youssef'),
    Etudiant('Imane'),
  ];

  classe[0].ajouterNote(15);
  classe[0].ajouterNote(17.5);
  classe[1].ajouterNote(9);
  classe[1].ajouterNote(12);
  classe[2].ajouterNote(11);
  classe[2].ajouterNote(14);
  classe[2].ajouterNote(13);
  classe[2].ajouterNote(25);   // refusée

  // ... suite : classement et statistiques
}
```

Sortie : (à ce stade)

```
Note invalide : 25.0
```

Remarque

La note 25 est refusée par `ajouterNote` : l'objet `Etudiant` **protège** ses données.

Mini-projet : le programme principal (2/2)

```dart
void main() {
  // ... création des étudiants et des notes

  classe.sort((a, b) => b.moyenne.compareTo(a.moyenne));

  print('--- Classement ---');
  for (final e in classe) {
    print(e);
  }

  double total = 0;
  for (final e in classe) {
    total += e.moyenne;
  }
  print('Moyenne de la classe : '
      '${(total / classe.length).toStringAsFixed(2)}');
  print('Major : ${classe.first.nom}');
}
```

Sortie : (programme complet)

```
Note invalide : 25.0
--- Classement ---
Sara : 16.25 (Très bien)
Imane : 12.67 (Assez bien)
Youssef : 10.50 (Passable)
Moyenne de la classe : 13.14
Major : Sara
```

Moyenne de la classe
$= (16{,}25 + 12{,}67 + 10{,}5)/3 \approx 13{,}14.$

Erreurs fréquentes du débutant

Erreur	Correction
Oubli du ;	Terminer chaque instruction par ;
= au lieu de == dans une condition	= affecte, == compare
`7 / 2` attendu entier	Utiliser `7 ~/ 2`
Index hors limites (`RangeError`)	Index de 0 à `length - 1`
Boucle infinie	Vérifier que la condition peut devenir fausse
Utiliser une variable `String?` comme `String`	Tester `null`, ou utiliser `??` / `?.`
Oublier `return` dans une fonction typée	Toute branche doit renvoyer une valeur
Oublier `super(...)` ou `this.`	Initialiser tous les attributs du constructeur

Récapitulatif du chapitre

Types et variables

`int`, `double`, `String`, `bool` ; `var`, `final`, `const` ; `null safety` (`?`, `??`, `?.`) ; `List`, `Set`, `Map`.

Conditions

`if / else if / else`, `switch`, ternaire `? :`

Boucles

`for`, `while`, `do-while`, `for-in`, `break`, `continue`, `forEach`.

Fonctions

Paramètres positionnels / nommés / optionnels, `=>`, fonctions anonymes, `map` / `where` / `reduce`, récursivité.

Classes

Attributs, méthodes, constructeurs (positionnels, nommés), encapsulation (`_`, getters/setters), `static`, `toString`, héritage, classes abstraites, polymorphisme, `enum`.

Pour aller plus loin

Prochaines notions

- Programmation **asynchrone** : `Future`, `async` / `await`, `Stream`
- Gestion des **exceptions** : `try` / `catch`
- Consommer une **API REST** (JSON, `http`)
- Interfaces Flutter : widgets et état

Ressources

- `dart.dev/language` : documentation officielle
- `dart.dev/codelabs` : tutoriels guidés
- `dartpad.dev` : éditeur en ligne
- `api.dart.dev` : référence de l'API

Conseil final

On apprend à programmer en **programmant** : refaites chaque exemple sans regarder la solution !